Explainable AI
Self-Interpretable Models

Project Handout (Revision 2)

1 Introduction

Deep Neural Networks are pushing state-of-the-art closer and closer to human performance on perception problem. However, their modes of failure are often different from those of humans. For example, it has been noted by Goodfellow et al. [2], that a clear image of a panda bear with just a tiny bit of human-imperceptible noise makes GoogLeNet confidently predict the "gibbon".

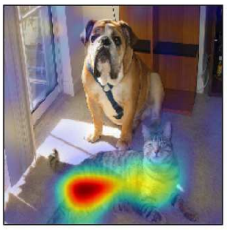
Understanding modes of failure, and, more generally, the reasoning of machine learning models is critical. This is especially true in safety-critical domains like autonomous driving, where redundancy needs to be built into systems and control may be handed down to humans in degraded conditions. A relatively new subfield of machine learning called *explainable AI* is concerned with methods and tools for understanding and interpreting predictions made by machine learning models.

Broadly speaking, explainable AI methods can be split into two main categories, based on their amount of "intrusiveness" into the model design and training process:

- *post-hoc analysis*: Methods in this class treat a pre-trained neural network as a *black box*. Given this setup, they aim to investigate its behavior. For example, grad-CAM [5] aims to re-construct the part of the image that supports the decision of a neural network to predict "cat" (see Figure 1). Note that post-hoc analysis methods make few or no assumptions about the network architecture, and assume it is already fully trained (hence the name "post-hoc").

- *self-interpretable models*: Yet other researchers focus on a different setup. What if we can find neural network architectures and/or training methods that make models easy to interpret. For example, the model may be designed such that it finds prototypical object parts from its training set in the input image that support its prediction. For example, ProtoPNet[1], can explain why a certain bird belongs to a species by linking its beak, wing or foot to corresponding training image patches (see Figure 1).

In this assignment, you will be focusing on self-interpretable models. Your goal is to get familiar with several approaches, run experiments to identify their



(c) Grad-CAM 'Cat'

(i) Grad-CAM 'Dog'

Figure 1: Given a test image GradCAM[5] can point out the spatial support (c, i) for various image class predictions (image reproduced from [5]).

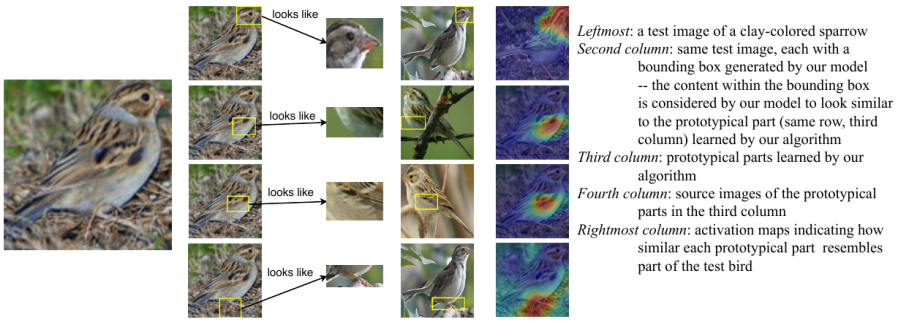


Figure 2: ProtoPNet[1] can justify a fine-grained classification decision by pointing out the similarity of object parts to prototypical patches from the training set (image reproduced from [1]).

strengths and drawbacks, and analyze the pros of cons of each. More specifically, you will get familiar with ProtoPNet[1], ProtoTree[4], TesNet[6] and PIPNet[3].

Your tasks are as follows:

- Literature survey: Read the relevant papers from the literature and understand their contributions, as well as their limitations (section 2.1).
- Performance Indicators: Decide what metrics you want to use for comparing the methods (section 2.2).
- Baseline: Implement your baseline method and run a baseline experiment, focusing on both accuracy and interpretability (section 2.3). The baseline is important, as it gives you a reference point with respect to which you can evaluate the methods.
- Method implementation: Write your own implementation of ProtoPNet, ProtoTree and TesNet in PyTorch (section 2.4).
- Experiments: Compare your methods against the baseline and against each other, under both accuracy and interpretability metrics. You most probably will need to tune hyperparameters (such as the learning rate or number of prototypes) to achieve good convergence and to improve the results (section 2.5).
- Documentation: Write a report summarizing your findings (section 2.6).

2 Steps

This section describes the steps you need to take to finish the project. You will most likely need to iterate between some of these steps (for example, run experiments and document results). Also, since this is a team project, some steps need to be done in parallel (*e.g.* each team member implements a method). Be careful with handling dependencies between steps when working together!

2.1 Step 1: Literature Survey

You should carefully read the three papers in chronological order ([1], [4] and [6]) to get familiar with the methods. It is good to keep a critical mindset when reading literature. For example:

- What are the computational implications of using the prototype layer in ProtoPNet compared to regular convolution?
- Why do the authors of ProtoPNet choose to make the prototypes class-specific? What is the drawback of this decision? Does ProtoTree and/or TesNet address this limitation?
- What happens if the prototypes are too large or too small? Can the methods find scale-invariant prototypes?

- How does on judge the interpretability of models? Could you devise some objective method for such a qualitative metric?
- How would you extend these methods to other problems, like object detection or semantic segmentation?
- Why do you think fine-grained classification problems have been chosen for evaluation by the authors of these papers?

2.2 Step 2: Performance Indicators

It is good to decide early on the relevant metrics. What you care about are three conflicting performance indicators:

- *Accuracy*: Clearly, interpretability should not significantly limit the accuracy of the method.
- *Interpretability*: This is a qualitative metric of how well the model explains its predictions.
- *Efficiency*: Accuracy and interpretability often come at the cost of more operations, or operations that are less hardware friendly. You should at least attend to the number of operations or network inference speed.

2.3 Step 3: Baseline

You will need a baseline before you start experimenting. You should decide on:

- *The task*: For this project, you should stick to classification, as it is still the dominant benchmark for comparing explainable methods in the literature. This will also simplify comparison with results reported in the papers.
- *The dataset*: You should aim to reproduce the results on the CUB-200 and Stanford Cars datasets.
- *The architecture*: You should choose a neural network architecture that is suitable for the amount of data you have. The VGG-16 and ResNet-34 backbone architectures used in the papers could provide a good starting point.

Once you have decided on the three elements above, you should train a classical classification network (backbone and fully-connected head). Make sure your network does not over-fit (use the validation set), and record the networks accuracy and efficiency, as discussed in section 2.2.

2.4 Step 4: Implementation

Now that you have your baseline, extend your codebase to support ProtoPNet, ProtoTree, TesNet and PIPNet, as described in the papers. Try to follow good software design practices and generalize the baseline – do not duplicate code, as this will be hard to maintain and error-prone. In the end, you should be able to configure your trainer to either train the baseline or the four target methods. The user should also be able to specify the dataset or any other hyperparameters. You can rely on publicly available code for inspiration, but note that you have to hand in your own Pytorch implementation.

2.5 Step 5: Experiments

Run training experiments with all three methods, on the same dataset and network architecture. You will want to experiment with changing hyperparameters, to get the best results, as defined by the performance indicators defined in section 2.2.

In addition, you should investigate the contribution critical hyperparameters, such as the number of prototypes and their sizes, or the height of the prototype tree.

Hint: Keep a detailed record of your experiments, as you will need to centralize and plot your results.

2.6 Step 6: Technical Report

You should now write a technical report about your findings. Your report should cover the following sections:

- *Problem*: What are you trying to accomplish?
- *Algorithms*: What methods are you experimenting with? Describe the methods briefly and try to relate them to each other. You can write a very short pseudocode for each. Make sure you cite the papers properly (this is very important when you do a conference submission!)
- *Experiments*: Describe your experimental procedure, including the baseline, experimental settings, hyperparameters and code.
- *Results*: Plot the performance/interpretations/efficiency of your methods as described in section 2.2 and summarize the relevant results in tables and figures. You might want to add other interesting plots, such as a prototype tree example.
- *Discussion*: Discuss your results. Be sure to comment on the merits/drawbacks of the methods, or any surprising results you might get.

Finally, using the material in the technical report, prepare a set of slides for your presentation.

References

[1] Chaofan Chen and Oscal Li and Chaofan Tao and Alina Jade Barnett and Jonathan Su and Cynthia Rudin. This Looks Like That: Deep Learning for Interpretable Image Recognition. In *Neural Information Processing Systems*, 2019.

[2] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarialexamples. In *International Conference on Learning Representations*, 2015.

[3] Meike Nauta, Jörg Schlötterer, Maurice van Keulen, and Christin Seifert. Pip-net: Patch-based intuitive prototypes for interpretable image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[4] Meike Nauta, Ron van Bree, and Christin Seifert. Neural Prototype Trees for Interpretable Fine-grained Image Recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[5] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In *International Conference on Computer Vision*, 2017.

[6] Jiaqi Wang, Huafeng Liu, Xinyue Wang, and Liping Jing. Interpretable Image Recognition by Constructing Transparent Embedding Space. In *International Conference on Computer Vision*, 2021.